

Além do RPC: Como Projeções e Cache Podem Transformar uma Plataforma de Seguros

Introdução

No artigo anterior exploramos como estruturar uma plataforma de seguros moderna utilizando Capabilities, Product Teams e Core Teams.

Vimos como domínios como:

- Pricing
- Produto
- Oferta
- Emissão
- Sinistro

podem evoluir de forma independente, mantendo autonomia entre equipes.

Porém, à medida que a plataforma cresce, surge um novo desafio:

Como reduzir a latência e o acoplamento entre sistemas sem criar uma arquitetura excessivamente complexa?

Muitas empresas tentam resolver esse problema substituindo REST por tecnologias mais rápidas como gRPC ou NATS RPC.

Embora isso traga ganhos de performance, existe uma pergunta mais importante:

Por que estamos fazendo chamadas remotas durante uma cotação?

Neste artigo veremos uma abordagem baseada em eventos, projeções e cache local que pode reduzir drasticamente a latência de uma plataforma de seguros.

O Problema

Imagine uma plataforma de Pricing responsável por calcular o prêmio de um seguro Auto.

Para realizar uma cotação, ela precisa consultar:

- Dados do cliente
- Dados do veículo
- Histórico de sinistros
- Regras do produto

Uma implementação tradicional seria:

Pricing

- |— Customer API
- |— Vehicle API
- |— Claims API
- |— Product API

Para cada cotação são realizadas diversas chamadas remotas.

Mesmo utilizando NATS RPC:

Pricing

- |— customer.get
- |— vehicle.get
- |— claims.get
- |— product.get

o problema continua existindo:

- Dependências síncronas
- Latência acumulada
- Timeouts
- Cascata de falhas
- Complexidade operacional

Otimizando a Comunicação

Muitas equipes começam por aqui.

Substituem:

REST

por:

gRPC

ou

NATS RPC

O resultado normalmente é:

REST $\approx$ 5-20 ms

NATS RPC $\approx$ 1-5 ms

Existe ganho.

Mas a dependência continua existindo.

A pergunta correta passa a ser:

Como eliminar chamadas desnecessárias?

Mudando a Mentalidade

Em vez de perguntar:

Como o Pricing consulta o Vehicle?

podemos perguntar:

Como o Pricing mantém localmente os dados necessários para cotação?

Essa mudança de perspectiva leva a uma arquitetura muito diferente.

O Conceito de Projeção

Cada capability publica eventos do seu domínio.

Exemplo:

Customer Platform

```
await publish(  
  "customer.updated",  
  {  
    "customer_id": "123",  
    "birth_date": "1990-01-01",  
    "gender": "M"  
  })
```

```
}  
)
```

Vehicle Platform

```
await publish(  
  "vehicle.updated",  
  {  
    "vehicle_id": "V123",  
    "fiipe": 85000,  
    "year": 2023,  
    "type": "SUV"  
  }  
)
```

Claims Platform

```
await publish(  
  "claims.updated",  
  {  
    "customer_id": "123",  
    "claims_count": 2  
  }  
)
```

Esses eventos são enviados para um barramento.

NATS + JetStream

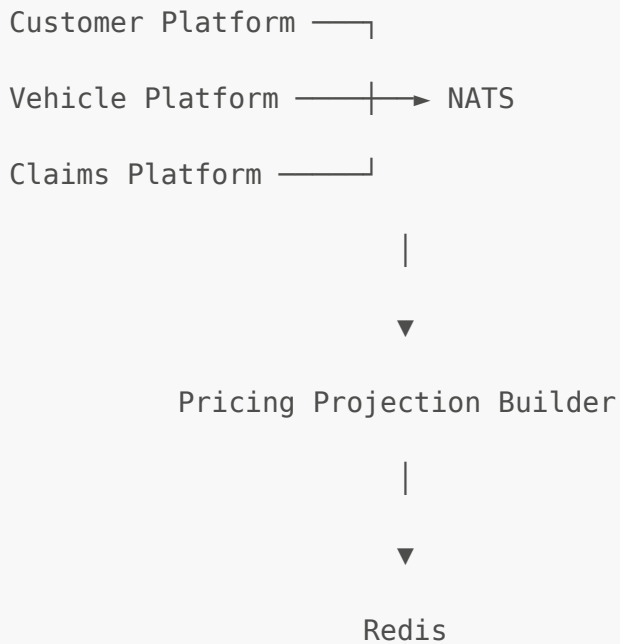
Quem Consome Esses Eventos?

O próprio Pricing.

Mais especificamente:

Pricing Projection Builder

Visualmente:



O Que é uma Projeção?

Uma projeção é uma visão dos dados otimizada para um caso de uso específico.

Por exemplo:

O Vehicle Service possui um documento enorme.

```
{
  "vehicle_id": "V123",
  "plate": "ABC1234",
  "documents": [...],
  "inspections": [...],
  "photos": [...],
  "claims": [...],
  "fipe": 85000,
  "year": 2023,
  "type": "SUV"
}
```

O Pricing não precisa disso tudo.

Ele precisa apenas de:

```
{
  "vehicle_fipe": 85000,
}
```

```
"vehicle_year": 2023  
}
```

Construindo a Projeção

Suponha que o Pricing precise apenas dos seguintes atributos:

```
@dataclass  
class PricingProjection:  
  
    customer_age: int  
  
    claims_count: int  
  
    vehicle_fipe: float  
  
    vehicle_year: int  
  
    risk_factor: float
```

O Projection Builder é responsável por montar esse objeto.

Evento de Cliente

```
projection.customer_age = 35
```

Evento de Sinistro

```
projection.claims_count = 2  
projection.risk_factor = 1.15
```

Evento de Veículo

```
projection.vehicle_fipe = 85000  
projection.vehicle_year = 2023
```

Resultado:

```
{  
  "customer_age": 35,  
  "claims_count": 2,  
  "vehicle_fipe": 85000,  
  "vehicle_year": 2023,  
  "risk_factor": 1.15  
}
```

Como o Pipeline Fica

Sem projeção:

Pricing

- └ Customer RPC
- └ Vehicle RPC
- └ Claims RPC
- └ Product RPC

Com projeção:

Pricing

|

▼

Redis

|

▼

Pipeline

|

▼

Pacote de Cálculo

Exemplo Real de Pipeline

```
class LoadProjectionStep:

    async def execute(self, ctx):

        projection = await redis.get(
            f"pricing:{ctx.customer_id}"
        )

        ctx.data["projection"] = projection
```

Etapas de Risco

```
class AutoRiskStep:

    async def execute(self, ctx):

        p = ctx.data["projection"]

        score = (
            p["risk_factor"]
            * p["vehicle_fipe"]
            / 10000
        )

        ctx.data["risk_score"] = score
```

Pacote de Cálculo

```
class PricingPackageStep:

    async def execute(self, ctx):

        projection = ctx.data["projection"]

        payload = {

            "age": projection["customer_age"],

            "claims": projection["claims_count"],

            "fipe": projection["vehicle_fipe"],

            "risk": ctx.data["risk_score"]
        }

        return await pricing_package.calculate(
```



```
)    payload
```

Nenhuma chamada remota.

Nenhum RPC.

Nenhum REST.

E Se o Cache Não Tiver o Dado?

Existem duas estratégias.

Estratégia 1 - Fallback

```
projection = await redis.get(key)

if projection is None:
    projection = await customer_rpc.get(
        customer_id
    )

    await redis.set(
        key,
        projection
    )
```

Mais simples.

Mais segura.

Estratégia 2 - Projection Only

```
projection = await redis.get(key)

if projection is None:
    raise ProjectionNotFound()
```

Mais comum em plataformas de alta escala.

Força a qualidade das projeções.

Um Insight Importante

Muitas empresas tentam compartilhar APIs.

Por exemplo:

Pricing → Customer API

Pricing → Vehicle API

Pricing → Claims API

Mas sistemas altamente escaláveis normalmente compartilham dados e não serviços.

Em vez disso:

Customer

|



Eventos

|



Pricing Projection

O acoplamento diminui drasticamente.

CQRS na Prática

Sem perceber, chegamos muito próximos de CQRS.

Temos:

Write Model

Responsável por atualizar os dados.

Customer

Vehicle

Claims

E:

Read Model

Responsável por consultas especializadas.

Pricing Projection

Offer Projection

Eligibility Projection

Cada capability mantém sua própria visão do negócio.

Benefícios

Essa arquitetura traz diversos ganhos:

Menor latência

Leitura local em Redis:

< 1 ms

Menor acoplamento

Pricing não depende de APIs externas.

Escalabilidade

Menos tráfego entre serviços.

Autonomia das equipes

Cada capability constrói suas próprias projeções.

Evolução independente

Novos campos podem ser adicionados sem alterar sistemas de origem.

Conclusão

Quando falamos de plataformas de seguros, a principal preocupação normalmente é a comunicação entre sistemas.

Mas, em muitos casos, a melhor otimização não é utilizar um protocolo mais rápido.

É eliminar a necessidade de comunicação durante o processamento.

Ao combinar:

- NATS JetStream
- Eventos de domínio
- Projection Builders
- Redis
- Pipelines de Pricing

é possível construir plataformas capazes de processar grandes volumes de cotações com baixa latência, reduzindo dependências síncronas e aumentando a autonomia dos times.

Talvez a pergunta mais importante não seja:

Como o Pricing chama outros sistemas?

Mas sim:

Como o Pricing mantém localmente apenas os dados necessários para tomar decisões?